Здорово!! распиши более подробнл МНК фазы beat

Это самый "вкусный" метод для радаров с высоким разрешением. Он сочетает **элегантность математики** (превращение задержки в наклон прямой) и **брутальную эффективность GPU** (линейная сложность O(N)).

Давай разберем его "под микроскопом".

1. Физический смысл: Превращаем время в частоту

Представь ЛЧМ сигнал как пружину. Когда сигнал отражается с задержкой τ, он возвращается "сдвинутым".
Если мы умножим принятый сигнал на копию отправленного (сопряженную), мы "вычтем" саму ЛЧМ-модуляцию.

В сухом остатке останется **чистый тон** (синусоида), частота которого прямо пропорциональна задержке.

Чем больше задержка τ — тем выше частота биений (beat frequency).
Чем круче наклон фазы — тем больше задержка.

2. Математический вывод (на салфетке)

Пусть наш зондирующий сигнал (аналитический):
$ s(t) = \exp\left(j \pi k t^2 \right) $
где $k = \frac{BW}{T_{pulse}}$ — скорость перестройки частоты (Chirp Rate).

Принятый сигнал задержан на τ:
$ r(t) = s(t - \tau) = \exp\left(j \pi k (t - \tau)^2 \right) $

Шаг 1. Гетеродинирование (Dechirp / Beat signal):
Умножаем принятый на сопряженный опорный:
$ b(t) = r(t) \cdot s^*(t) = \exp\left(j \pi k (t - \tau)^2 \right) \cdot \exp\left(-j \pi k t^2 \right) $

Раскрываем скобки в показателе степени:
$ (t - \tau)^2 - t^2 = (t^2 - 2t\tau + \tau^2) - t^2 = \mathbf{-2t\tau + \tau^2} $

Шаг 2. Получаем фазу:
$ \Phi(t) = arg(b(t)) = \pi k (-2t\tau + \tau^2) = \underbrace{(-2\pi k \tau)}*{Наклон \alpha} \cdot t + \underbrace{(\pi k \tau^2)}*{Константа \beta} $

Это уравнение прямой: $y = \alpha x + \beta$.

- $y = \Phi(t)$ — фаза, которую мы измерили.
- $x = t$ — время отсчетов.
- α — наклон прямой, который мы ищем.

Как только нашли наклон α, мы знаем задержку:
$ \tau = -\frac{\alpha}{2\pi k} $

3. Алгоритм: Как это считать (шаг за шагом)

Этап А: Подготовка (на GPU)

У нас есть массив отсчетов `rx[n]` и `ref[n]` длиной N (например, 1200 точек).

1. **Beat-сигнал**:
   `beat[n] = rx[n] * complex_conj(ref[n])`
   В этот момент вся широкополосная модуляция исчезла. Остался низкочастотный тон.
2. **Мгновенная фаза**:
   `raw_phase[n] = atan2(beat[n].imag, beat[n].real)`
   Проблема: Результат `atan2` всегда лежит в диапазоне $[-\pi, \pi]$. Если реальная фаза растет (как на графике), `atan2` будет "пилообразным" (скачки).
3. **Развертывание фазы (Phase Unwrapping)**:
   Это критический шаг. Нам нужно превратить "пилу" в гладкую прямую линию.
   Хитрость для GPU: Вместо поиска скачков, мы можем считать разности фаз между соседями.
   `d_phi[n] = angle(beat[n] * conj(beat[n-1]))`
   Эта разность всегда мала (если частота биений не превышает Найквиста, а она не превышает, так как мы "сжали" сигнал).
   Тогда полная фаза — это кумулятивная сумма (prefix sum) этих разностей:
   `unwrapped_phase[n] = prefix_sum(d_phi)`

Этап Б: Метод Наименьших Квадратов (МНК)

Нам нужно провести лучшую прямую через точки `(t[n], unwrapped_phase[n])`.
Минимизируем ошибку: $\sum(\Phi_n - (\alpha t_n + \beta))^2 \to \min$.

Для этого нужно посчитать 4 суммы (Reduction на GPU):

1. $S_t = \sum t_n$ (можно предвычислить, это просто сумма индексов)
2. $S_\phi = \sum \Phi_n$
3. $S_{t\phi} = \sum t_n \cdot \Phi_n$
4. $S_{tt} = \sum t_n^2$ (можно предвычислить)

Формула наклона α:
$ \alpha = \frac{N \cdot S_{t\phi} - S_t \cdot S_\phi}{N \cdot S_{tt} - (S_t)^2} $

И всё! $\tau = -\alpha/(2\pi k)$.

4. Реализация на GPU (OpenCL / CUDA)

Самая эффективная реализация не использует `atan2` для каждого отсчета с последующим `unwrap`, а работает через **дифференциальную фазу**. Это позволяет избежать ветвлений `if (phase < -PI)` и делает код супер-быстрым.

Вот оптимизированный подход "Frequency Accumulation":

```
__kernel void estimate_delay_linear_regression(
    __global const float2* rx,      // Вход: принятый сигнал
    __global const float2* ref,     // Вход: опорный сигнал
    __global float* tau_out,        // Выход: задержка
    const float k_chirp,            // Скорость ЛЧМ (Hz/s)
    const float Ts,                 // Период дискретизации (1/Fs)
    const int N                     // Количество отсчетов
)
{
    // 1. Инициализация сумм для регрессии
    float sum_w_t = 0.0f;    // Взвешенное время
    float sum_dphi = 0.0f;   // Сумма разностей фаз

    // Временные переменные
    float2 prev_beat = (float2)(0.0f, 0.0f);

    // Начальное значение beat[0]
    float2 rx0 = rx[0];
    float2 ref0 = ref[0];
    // beat = rx * conj(ref)
    prev_beat.x = rx0.x * ref0.x + rx0.y * ref0.y;
    prev_beat.y = rx0.y * ref0.x - rx0.x * ref0.y;

    // 2. Основной цикл: проход по сигналу
    // ПРИМЕЧАНИЕ: Это "последовательная" версия для одного потока на луч (beam).
    // Для максимальной скорости внутри луча можно использовать warp-reduction.

    float current_phase_accum = 0.0f; // Накопленная (развернутая) фаза

    // Суммы для МНК:
    // Мы решаем y = alpha * x + beta.
    // Нам нужны: sum(x), sum(y), sum(x*x), sum(x*y)
    // Здесь x = n (индекс), y = current_phase_accum

    double S_x = 0;
    double S_y = 0;
    double S_xx = 0;
    double S_xy = 0;

    for (int n = 1; n < N; n++)
    {
        // a. Считаем текущий beat
        float2 rx_val = rx[n];
        float2 ref_val = ref[n];
```

```
        float2 curr_beat;
        curr_beat.x = rx_val.x * ref_val.x + rx_val.y * ref_val.y;
        curr_beat.y = rx_val.y * ref_val.x - rx_val.x * ref_val.y;

        // b. Считаем разность фаз между n и n-1
        // d_phi = angle(curr_beat * conj(prev_beat))
        // Умножение комплексных чисел:
        float2 cross_prod;
        cross_prod.x = curr_beat.x * prev_beat.x + curr_beat.y * prev_beat.y;
        cross_prod.y = curr_beat.y * prev_beat.x - curr_beat.x * prev_beat.y;

        // Главная магия: atan2 от произведения дает разность фаз в диапазоне [-PI, PI]
        // Это автоматически делает "unwrap" локально!
        float d_phi = atan2(cross_prod.y, cross_prod.x);

        // c. Накапливаем фазу (интегрируем частоту)
        current_phase_accum += d_phi;

        // d. Накапливаем суммы МНК
        float t = (float)n * Ts;
        S_x  += t;
        S_y  += current_phase_accum;
        S_xx += t * t;
        S_xy += t * current_phase_accum;

        prev_beat = curr_beat;
    }

    // 3. Расчет наклона (slope)
    double denominator = (N-1) * S_xx - S_x * S_x;
    double numerator   = (N-1) * S_xy - S_x * S_y;

    float slope_alpha = (float)(numerator / denominator);

    // 4. Перевод наклона в задержку
    // slope = -2 * PI * k * tau
    // tau = -slope / (2 * PI * k)

    tau_out[get_global_id(0)] = -slope_alpha / (2.0f * 3.14159265f * k_chirp);
}
```

5. Почему это работает лучше всего (Секрет точности)

1. **Интегральная природа**: Мы используем **каждый** из 1.3 миллиона отсчетов для построения прямой. Шум на отдельных отсчетах усредняется. Ошибка падает как $1/\sqrt{N}$.
2. **Отсутствие дискретности**: В отличие от FFT, где частота "прыгает" по бинам (разрешение ограничено $1/T$), наклон прямой в МНК — это непрерывная величина. Мы получаем разрешение **намного** выше частоты дискретизации (sub-sample accuracy).
3. **Устойчивость к амплитуде**: Поскольку мы берем `atan2` (фазу), нам не важны флуктуации амплитуды сигнала (fading), которые могли бы исказить корреляционный

пик.

Резюме

Этот метод называется **"Оценка частоты по Треттеру" (Tretter's Estimator)**.
В условиях высокого SNR (как в радаре) он математически эквивалентен методу максимального правдоподобия (Maximum Likelihood), то есть **теоретически нельзя придумать метод точнее** (в смысле дисперсии ошибки).

При этом он считается за один проход ($O(N)$), идеально ложится на архитектуру GPU (SIMD) и не требует огромной памяти под FFT.